

JMLR Reproducibility Strategy — Before Reviewer Testing

This strategy is optimized for:

- Very old local hardware (Celeron)
 - Existing `run_all_checkpoint` already tested
 - Two execution modes:
 - FAST
 - PAPER_QUALITY
 - Mixed execution environment:
 - Local machine
 - GitHub Actions
 - Colab / Kaggle
 - Goal: finish PAPER_QUALITY reproducibility first, then prepare reviewer-safe execution.
-

Core Principle

Do **not** run everything in one environment.

Instead:

- Use local machine for lightweight or checkpoint-safe stages
 - Use Colab/Kaggle for heavy symbolic or long experiments
 - Use GitHub Actions for reproducibility validation only
 - Separate FAST and PAPER_QUALITY into independent execution paths
-

Recommended Execution Order

Phase 1 — Local Machine (Checkpoint Builder)

Use local hardware only for:

- dependency validation
- repo validation
- checkpoint creation
- lightweight experiments
- figure/table generation
- reproducibility hashing

Run:

```
python run_all.py --skip-slow
```

This creates:

- baseline checkpoint
- partial completed pipeline
- stable logs
- cached outputs

This is your reproducibility backbone.

Phase 2 — Heavy Jobs Offloaded to Colab/Kaggle

Heavy experiments should never run on old Celeron.

Move only unfinished heavy stages.

Recommended heavy modules:

Step	Environment	Reason
exp2	Colab/Kaggle	symbolic regression heavy
suppB	Colab/Kaggle	noise sweep large runtime
instability	Colab/Kaggle	repeated seeds / K loops
extrapolation sweeps	Colab/Kaggle	CPU-heavy

Use:

```
python run_all.py --resume --only exp2
python run_all.py --resume --only suppB
python run_all.py --resume --only instability
```

Run each separately.

Never launch all slow experiments together.

Phase 3 — Merge Checkpoint Back Locally

After Colab/Kaggle finishes:

1. Download updated checkpoint
2. Replace local checkpoint
3. Resume locally

```
python run_all.py --resume
```

This should complete:

- tables
 - figures
 - manifests
 - hashes
 - provenance logs
-

GitHub Actions Strategy (Critical)

GitHub Actions should NOT become your main execution engine.

Use GitHub Actions only for:

- reviewer trust
 - clean environment validation
 - smoke tests
 - minimal reproducibility verification
-

Recommended GitHub Actions Split

Version A — FAST CI

Purpose:

- pull request validation
- sanity checks
- fast reproducibility

Runtime target:

- under 20–30 minutes

Recommended CI:

```
FAST MODE:  
- install dependencies  
- validate imports  
- run minimal experiments  
- run exp3 only  
- run one seed  
- validate figures  
- verify checkpoint schema
```

Use:

```
python run_all.py --fast
```

or

```
python run_all.py --skip-slow
```

Version B — PAPER_QUALITY CI

Purpose:

- archive reproducibility
- final release validation
- JMLR artifact confidence

Runtime target:

- 2–6 hours

Recommended trigger:

- manual workflow dispatch
- tagged release
- pre-submission execution

Recommended CI:

PAPER QUALITY:

- restore cached checkpoint
- run unfinished heavy stages
- generate tables
- regenerate figures
- compare hashes
- validate metrics
- package artifacts

Never run PAPER_QUALITY on every commit.

Colab vs Kaggle Recommendation

Colab

Best for:

- rapid testing
- exp2
- debugging
- one experiment at a time

Weakness:

- disconnect risk
-

Kaggle

Best for:

- long uninterrupted jobs
- stability
- large symbolic runs
- overnight execution

Recommended:

Use Kaggle for PAPER_QUALITY heavy jobs.

Use Colab for FAST experimentation.

Recommended Folder Strategy

```
logs/  
  pipeline_checkpoint.json  
  
results/  
  fast/  
  paper_quality/  
  
artifacts/  
  reviewer/  
  release/
```

This prevents collisions.

Reviewer Strategy (Very Important)

You should prepare two reviewer modes.

Reviewer Fast Mode

Purpose:

- reviewer confirms pipeline works
- low compute burden

Expected runtime:

- 20–60 minutes

Use:

```
python reproduce_local.py --fast
```

Reviewer Paper Mode

Purpose:

- reviewer reproduces final paper

Use:

```
python reproduce_local.py --paper
```

Best Execution Sequence

Day 1

Local:

```
python run_all.py --skip-slow
```

Day 2

Colab/Kaggle:

```
python run_all.py --resume --only exp2
python run_all.py --resume --only suppB
python run_all.py --resume --only instability
```

Day 3

Local merge:

```
python run_all.py --resume
```

Day 4

GitHub Actions:

- FAST CI
 - PAPER_QUALITY CI
-

Highest Priority Recommendation

For your hardware constraints, the safest path is:

1. Build checkpoint locally
2. Finish heavy runs on Kaggle/Colab
3. Merge locally
4. Freeze outputs
5. Validate through GitHub Actions
6. Only then move to reviewer packaging

This minimizes:

- wasted compute
 - restart failures
 - CI timeout risk
 - reviewer frustration
 - reproducibility instability
-

Most Important Rule

Never force one machine to run everything.

The checkpoint system is your strongest advantage.

Use checkpoint as distributed orchestration across environments.

This is the safest strategy for JMLR-level reproducibility.